



华中科技大学

计算机系统基础

子程序设计、函数调用与返回的机理

许 向 阳

xuxy@hust.edu.cn

《x86汇编语言程序设计》第8章，许向阳，华科大学出版社，2020年





第8章 子程序设计

8.1 子程序的概念

8.2 子程序的基本用法

子程序的定义、调用和返回

参数传递、现场保护

8.3 子程序应用示例

8.4 C语言程序中函数运行机理





课堂目标

- 能够准确地描述 **CALL**、**RET**指令的执行过程；
- 能够描述有哪些参数的传递方法，各有什么优缺点；
- 能够画出**栈帧示意图**，指出参数、局部变量在栈帧的位置；
- 能够给出参数、局部变量的**空间分配与回收时机**；
- 能够解释C语言中函数调用（包括参数个数不定、参数缺省值、传值、传址等）和函数返回的实现机理；
- 能够描述**缓冲区溢出攻击**的基本原理；
- 能够给出防范缓冲区溢出攻击的措施。





重要概念

ABI: Application Binary Interface
应用二进制接口

ABI 是计算机程序在二进制层面的交互规范，定义了编译后的**机器码**如何与**操作系统**、**库函数**或其他**程序模块**正确**通信**。它是软件组件间的“底层契约”，确保不同模块编译后能无缝链接与运行。





重要概念

ABI 规范了程序编译为机器码后，在内存和处理器层面的所有交互细节。

数据布局、函数调用约定、系统调用规范、二进制文件格式

➤ 数据布局

- 基本类型（int、float、指针）的字节大小
- 结构体 / 数组的内存对齐规则（对齐字节数）
- 字节序（大端 / 小端）





重要概念

➤ 函数调用约定 (Calling Convention)

- 参数传递：用寄存器？用栈？ 入栈顺序（从左到右 / 从右到左）
- 谁清理栈（调用者 / 被调用者）
- 返回值存放位置（寄存器 / 栈）
- 哪些寄存器必须被保存 (callee-saved)



重要概念

➤ 系统调用规范

- 系统调用号、中断 / 异常入口

- 内核态参数传递方式

➤ 二进制文件格式

- 可执行文件 / 库格式: ELF (Linux)、PE (Windows)、Mach-O (macOS)

- 符号命名规则 (如 C++ 名称修饰 name mangling)

➤ 其他

- 异常处理、线程局部存储 (TLS) 布局





重要概念

C++ 无统一标准 ABI

不同编译器（GCC/Clang/MSVC）二进制不兼容

同一个编译器而用不同的编译开关，结果也不相同

PPT 中的内容，或者书上的内容，只是一些示例，

不代表一定是这样的！无需死记硬背！

理解具体的实现细节时，以当前的机器语言程序为准





重要概念

Q: ISA 和 ABI 有什么差别 ?

Instruction Set Architecture, 指令集架构

Application Binary Interface, 应用二进制接口

ISA 管 CPU 认不认指令;

ABI 管程序之间 / 程序与系统能不能顺畅调用。





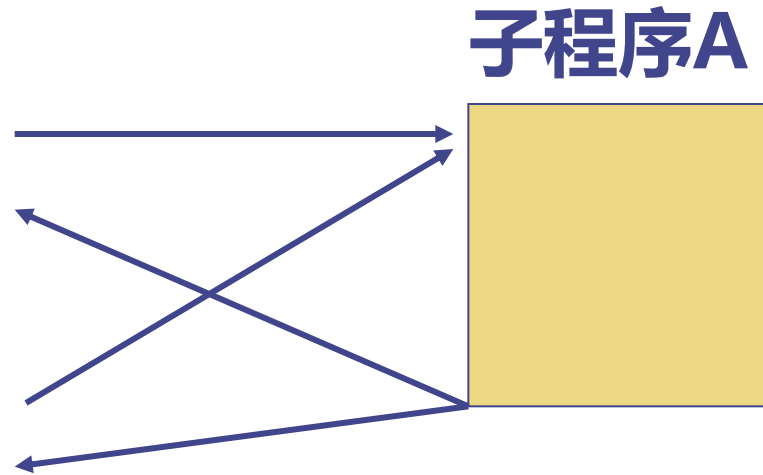
華中科技大學

8.1 子程序的概念



8.1 子程序的概念

K : 调用子程序 A
DK:
.....
J : 调用子程序 A
DJ:
.....



思考:

转移的本质是什么?

改变 EIP/RIP

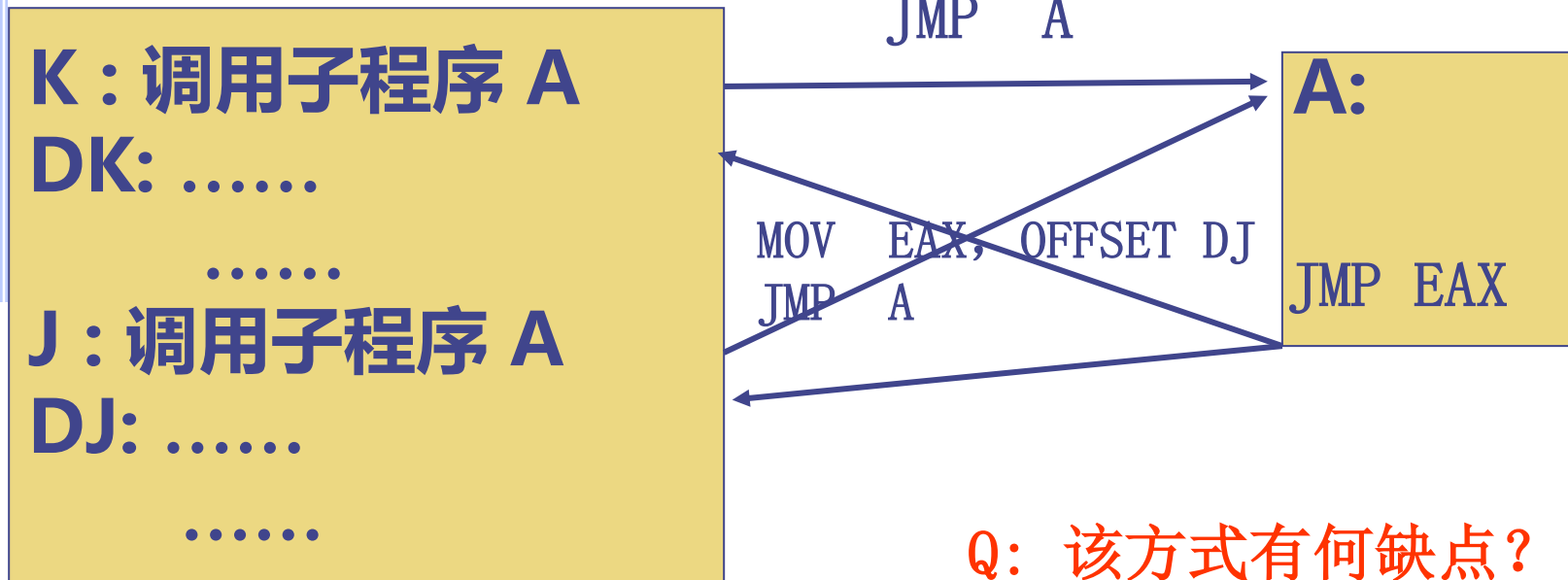
断点: 转子指令的直接后继指令的地址。

子程序执行完毕, 返回主程序的断点处继续执行

如何改变 EIP/RIP, 使其按照预定的路线图前进?

8.1 子程序的概念

使用已学过的指令完成转移



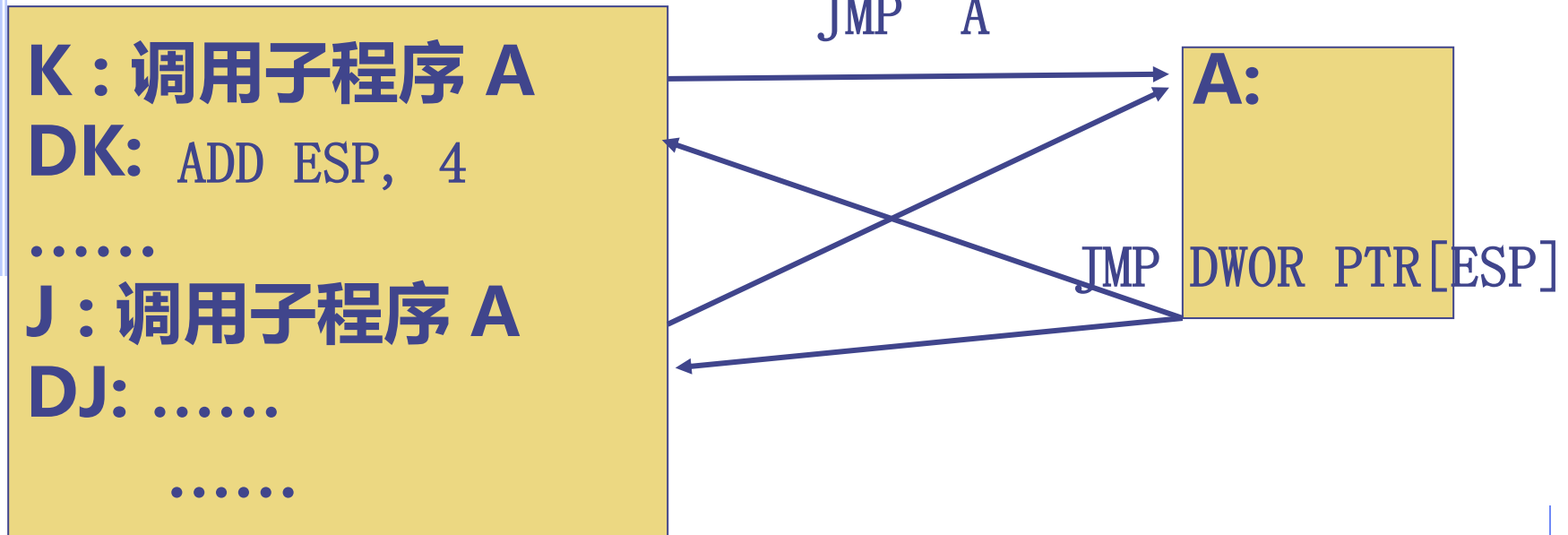
Q: 该方式有何缺点?

- (1) 使用了寄存器 EAX, 在子程序结束时, 要保证EAX与进入子程序时的值一样;
- (2) 在主程序中, 多引入了标号。



8.1 子程序的概念

使用已学过的指令完成转移



Q: 该方式有何缺点?

- (1) 在主程序中, 多引入了标号;
- (2) 要编程者控制 ESP。





8.2 子程序的基本用法

函数的调用与返回

调用指令

直接 **CALL** 函数名

间接 **CALL** DWORD PTR OPD

CPU 要做的工作:

(1) 保存断点

(EIP) $\rightarrow$ ↓ (ESP)





8.2 子程序的基本用法

子程序的调用与返回

(1) 保存断点

ESP →

断点的EA

(2) 将子程序的地址送 EIP

注意：间接调用获取地址的方式。





8.2 子程序的基本用法

子程序的调用与返回

返回指令： RET

返回指令的执行过程：

返回： $\uparrow (\text{ESP}) \rightarrow \text{EIP}$

特别注意： 栈顶应该是主程序的断点地址，
否则，运行可能会出现问題。





8.2 子程序的基本用法

子程序的调用与返回

返回指令: `RET n`

消除不再使用的入口参数对堆栈空间的占用

STEP 1:

$\uparrow (\text{ESP}) \rightarrow \text{EIP}$

STEP 2:

$(\text{ESP}) + n \rightarrow \text{ESP}$

`stdcall` 采用在子程序中恢复堆栈的方法





8.2 子程序的基本用法

Q: 子程序中要使用堆栈，即改变 ESP，怎么办？

注意：ret是从当前的栈顶弹出一个双字送给EIP。

在刚进入子程序时，有：

```
push  ebp
mov   ebp, esp
```

之后保持ebp不变，改变esp

.....

返回前，恢复esp

```
mov   esp, ebp
pop   ebp
ret
```

ESP →
EBP ↗

原 ebp
断点的EA

ESP →

EBP →

原 ebp
断点的EA





8.3 子程序应用示例

```
main proc c
```

```
    CALL DISPLAY
```

```
    DB 'Very Good', 0DH, 0AH, 0
```

```
    CALL DISPLAY
```

```
    db '12345', 0DH, 0AH, 0
```

```
    invoke ExitProcess, 0
```

```
main endp
```

```
DISPLAY PROC
```

```
    .....
```

```
DISPLAY ENDP
```

```
END
```

Q: 如何得到串的首地址?

编写子程序，使其能够显示**CALL**指令下面的串

Q: 断点在何处？子程序运行后返回到何处？

Q: 如何显示字符串？

```
putchar proto c :byte  
显示给定ASCII对应的字符
```

```
printf( "...");
```





8.3 子程序应用示例

CALL DISPLAY

msg1 DB 'Very Good', 0DH, 0AH, 0

CALL DISPLAY

msg2 db '12345', 0DH, 0AH, 0

Q: 生成的机器代码是什么样的？

```

008A2040 E8 20 00 00 00 call display (08A2065h)
008A2045 56          push     esi
008A2046 65 72 79      jb       _display@0+5Dh (08A20C2h)
008A2049 20 47 6F      and      byte ptr [edi+6Fh],al
008A204C 6F          outs     dx,dword ptr [esi]
008A204D 64 0D 0A 00 E8 0F or      eax,0FE8000Ah
.....
008A2051 EB 0F 00 00 00
008A2056 .....
008A2065 5B          pop      ebx
    
```

Q: 第一个CALL的机器码中， 20 00 00 00 20， 是什么含义？

008A2065 - 008A2045 = 00 00 00 20

Q: 56 65 72 79 20 47 6F 6F 64 0D 0A 00， 是什么含义？





8.3 子程序应用示例

```
.686P
.model flat, stdcall
ExitProcess proto :dword
includelib kernel32.lib
putchar      proto c :byte ;显示给定 ASCII 对应的字符
includelib libcmt.lib
includelib legacy_stdio_definitions.lib
.stack 200
.code
main proc c
    call display
    msg1 db 'Very Good', 0DH, 0AH, 0
    call display
    msg2 db '12345', 0DH, 0AH, 0
    invoke ExitProcess, 0
main endp
```





8.3 子程序应用示例

Q: 如何得到串的首地址？子程序运行后如何返回到希望的位置？

```
display proc
    pop    ebx
p1:
    cmp    byte ptr [ebx], 0
    je     exit
    invoke putchar, byte ptr [ebx]
    inc    ebx
    jmp    p1
exit:
    inc    ebx
    push   ebx
    ret
display endp
end
```

自我修改返回地址的子程序

Q1: 若将 ExitProcess
移到子程序之后，结果
如何？

Q2: 若少写RET之前的
INC EBX，结果如何？





8.3 子程序应用示例

自我修改程序

在程序中，将数据段的一段数据拷贝到代码段，
让程序运行这段数据对应的代码。

```
.data
```

```
machine_code db 0E8H, 20H, 0, 0, 0
```

```
db 'Very Good', 0DH, 0AH, 0
```

```
db 0E8H, 0FH, 0, 0, 0
```

```
db '12345', 0DH, 0AH, 0
```

```
len = $-machine_code
```

```
oldprotect dd ?
```





8.3 子程序应用示例

. 686P

.model flat, stdcall

ExitProcess proto :dword

VirtualProtect proto :dword, :dword, :dword, :dword

includelib kernel32.lib

putchar proto c :byte ; 显示给定 ASCII 对应的字符

includelib libcmtd.lib

includelib legacy_stdio_definitions.lib

.stack 200





8.3 子程序应用示例

```
.code
main    proc    c
        mov     eax, len
        mov     ebx, 40H
        lea     ecx, CopyHere
        invoke  VirtualProtect, ecx, eax, ebx, offset oldprotect
        mov     ecx, len
        mov     edi, offset CopyHere
        mov     esi, offset machine_code
```



8.3 子程序应用示例

CopyCode:

```
mov  al, [esi]
mov  [edi], al
inc  esi
inc  edi
loop CopyCode
```

CopyHere:

```
db  len dup(0)
invoke ExitProcess, 0
main  endp
```





8.4 C语言程序中函数的运行机理





8.4 C语言程序中函数的运行机理

- 如何传递参数？
- 传递什么？
 - 按值传递、按地址传递、按引用传递
 - 不同类型的形参/实参， 传递的内容有何差别？
- 传到什么地方去了？
- 如何进入函数？
- 如何从函数返回？
- 如何传递函数返回值？
- 函数中变量空间如何分配？
- 如何理解递归函数调用？





8.4 C语言程序中函数的运行机理

```
#include <stdio.h>
int fadd(int x, int y)
{
    int u, v, w;
    u=x+10;
    v=y+25;
    w=u+v;
    return w;
}
```

```
int main( )
{
    int a=100;    // 0x 64
    int b=200;    // 0x C8
    int sum=0;
    sum=fadd(a, b);
    printf("%d\n", sum);
    return 0;
}
```





8.4 C语言程序中函数的运行机理

变量空间分配

```
13:      int  a=100;      // 0x 64
00401088      mov     dword ptr [ebp-4], 64h
14:      int  b=200;      // 0x C8
0040108F      mov     dword ptr [ebp-8], 0C8h
15:      int  sum=0;
00401096      mov     dword ptr [ebp-0Ch], 0
16:      sum=fadd(a, b);
```

in(int, char **)		Name	Value
	Value	&a, x	0x0012ff7c
	100	&b, x	0x0012ff78
	200	&sum, x	0x0012ff74
	0	ebp, x	0x0012ff80

0012FF74 00 00 00 00 C8 00 00 00 64 00 00 00



8.4 C语言程序中函数的运行机理

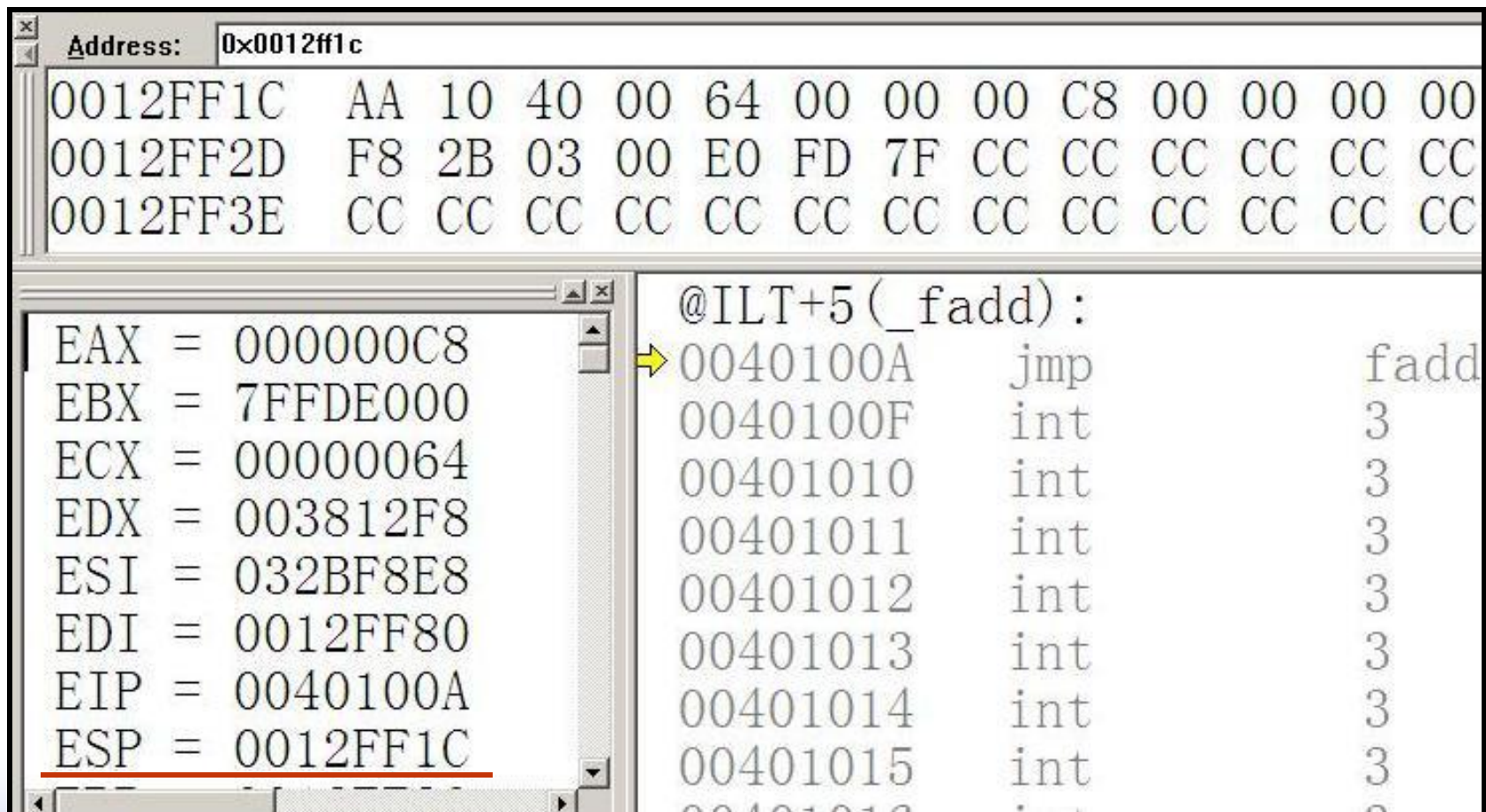
函数调用

```
13:      int  a=100;      // 0x 64
00401088      mov          dword ptr [ebp-4], 64h
14:      int  b=200;      // 0x C8
0040108F      mov          dword ptr [ebp-8], 0C8h
15:      int  sum=0;
00401096      mov          dword ptr [ebp-0Ch], 0
16:      sum=fadd(a, b);
● 0040109D      mov          eax, dword ptr [ebp-8]
➔ 004010A0      push         eax
004010A1      mov          ecx, dword ptr [ebp-4]
004010A4      push         ecx
004010A5      call          @ILT+5(_fadd) (0040100a)
004010AA      add          esp, 8
004010AD      mov          dword ptr [ebp-0Ch], eax
```


8.4 C语言程序中函数的运行机理

Sum=fadd(a,b)

执行CALL指令后的状态



The screenshot shows a debugger window with the following content:

Address: 0x0012ff1c

Address	Hex	Hex	Hex	Hex	Hex	Hex	Hex	Hex	Hex	Hex	Hex	Hex	Hex
0012FF1C	AA	10	40	00	64	00	00	00	C8	00	00	00	00
0012FF2D	F8	2B	03	00	E0	FD	7F	CC	CC	CC	CC	CC	CC
0012FF3E	CC	CC	CC	CC	CC	CC	CC	CC	CC	CC	CC	CC	CC

Registers:

- EAX = 000000C8
- EBX = 7FFDE000
- ECX = 00000064
- EDX = 003812F8
- ESI = 032BF8E8
- EDI = 0012FF80
- EIP = 0040100A
- ESP = 0012FF1C

Disassembly:

@ILT+5(_fadd):

- 0040100A jmp fadd
- 0040100F int 3
- 00401010 int 3
- 00401011 int 3
- 00401012 int 3
- 00401013 int 3
- 00401014 int 3
- 00401015 int 3



8.4 C语言程序中函数的运行机理

Sum=fadd(a,b)

执行CALL指令后的状态

```
004010A5  call    @ILT+5(_fadd) (0040100a)
004010AA  add     esp,8
```

0012FF1C

ESP →

断点地址

EIP 为函数的入口地址

(EIP) = 0040100A

a的值(即100)压栈

b的值(即200)压栈

0012FF1C AA 10 40 00 64 00 00 00 C8 00 00 00



8.4 C语言程序中函数的运行机理

```

3:  int fadd(int x, int y)
4:  {
00401020  push    ebp
00401021  mov     ebp,esp
00401023  sub     esp,4Ch
00401026  push    ebx
00401027  push    esi
00401028  push    edi
00401029  lea     edi,[ebp-4Ch]
0040102C  mov     ecx,13h
00401031  mov     eax,0CCCCCCCCh
00401036  rep stos dword ptr [edi]
5:      int u,v,w;
6:      u=x+10;
00401038  mov     eax,dword ptr [ebp+8]
0040103B  add     eax,0Ah
0040103E  mov     dword ptr [ebp-4],eax
    
```



观察形参*x*的位置

8.4 C语言程序中函数的运行机理

```
5:   int u,v,w;
6:   u=x+10;
   mov     eax,dword ptr [ebp+8]
   add     eax,0Ah
   mov     dword ptr [ebp-4],eax
7:   v=v+25;
   mov     ecx,dword ptr [ebp+0Ch]
   add     ecx,19h
   mov     dword ptr [ebp-8],ecx
8:   w=u+v;
   mov     edx,dword ptr [ebp-4]
   add     edx,dword ptr [ebp-8]
   mov     dword ptr [ebp-0Ch],edx
9:   return w;
   mov     eax,dword ptr [ebp-0Ch]
```



观察局部变量的位置

8.4 C语言程序中函数的运行机理

函数调用——返回

```

9:      return w;
mov     eax,dword ptr [ebp-0Ch]
10:  }
pop     edi
pop     esi
pop     ebx
mov     esp, ebp
pop     ebp
ret
    
```



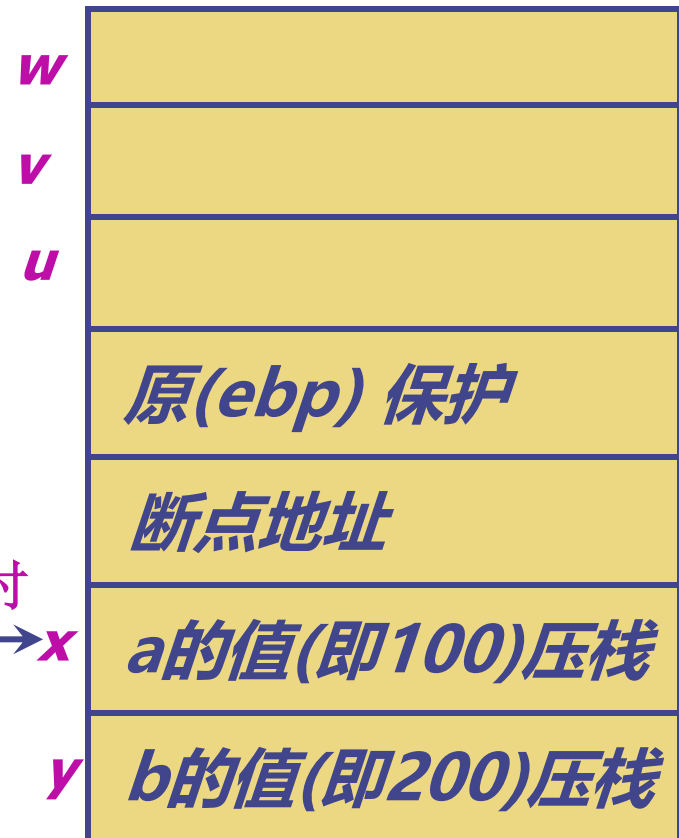
思考：局部变量的作用域？
 局部空间的释放？
 函数如何的返回？
 改变形参的值，对实参有影响吗？

8.4 C语言程序中函数的运行机理

```

15:      int sum=0;
mov     dword ptr [ebp-0Ch],0
16:      sum=fadd(a,b);
mov     eax,dword ptr [ebp-8]
push    eax
mov     ecx,dword ptr [ebp-4]
push    ecx
call    @ILT+5(_fadd)
➔ add     esp,8
mov     dword ptr [ebp-0Ch],eax
    
```

ret 返回时
esp → *x*





8.4 C语言程序中函数的运行机理

```
17:      printf("%d\n", sum);
004010B0      mov          edx, dword ptr [ebp-0Ch]
004010B3      push         edx
004010B4      push         offset string "%d\n" (0042201c)
004010B9      call          printf (004010f0)
004010BE      add          esp, 8
18:      return 0;
004010C1      xor          eax, eax
19:      }
004010C3      pop          edi
004010C4      pop          esi
004010C5      pop          ebx
004010C6      add          esp, 4Ch
004010C9      cmp          ebp, esp
004010CB      call          __chkesp (00401170)
004010D0      mov          esp, ebp
004010D2      pop          ebp
004010D3      ret
```





小结

函数调用语句:

参数压栈

断点地址保存

函数入口地址 $\rightarrow$ EIP

进入函数: {

(ebp) 保护

其他寄存器的保护

局部变量登上舞台

(自动分配空间)

函数返回: return 语句

寄存器恢复

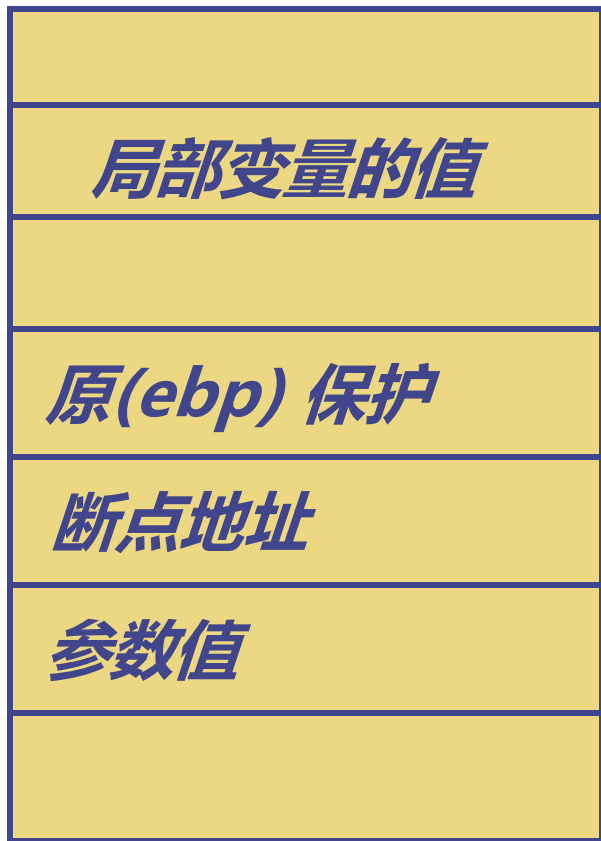
(ebp) 恢复

弹出断点地址 $\rightarrow$ EIP

局部变量退出舞台 (空间中内容不变)

局部变
量的地
址

形参的
地址



Q1: 如何理解：局部变量的空间分配、空间中内容初始化、空间释放？

```
int x=10;    mov dword ptr[ebp-...], 0AH
```

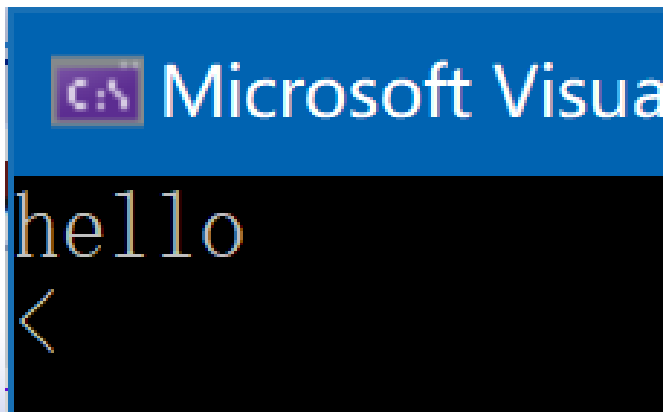
Q2: 在 C++中，为什么说 对象的构造，并不负责对象本身的空间分配，而只是完成空间中内容的初始化工作？

Q3: 在 C++中，为什么说 对象的析构，并不负责对象本身的空间释放，而只是完成一些其他的工作？



小测验

```
#include <stdio.h>
char* f()
{
    char temp[20];
    strcpy_s(temp, "hello");
    return temp;
}
```



```
int main()
{
    char* p;
    char a[20];
    int i=0;
    p = f();
    while (*(p + i) != 0) {
        a[i] = p[i];
        i++;
    }
    a[i] = 0;
    printf("%s \n", a);
    printf("%s \n", p);
    return 0;
}
```

i
a[20]
p

temp[20]
.....
i
a[20]
p

printf传入
的参数
.....
i
a[20]
p

小测验

warning C4172: 返回局部变量或临时变量的地址: temp

```
a[i] = 0;  
printf("%s \n", a);  
printf("%s \n", p);  
return 0;
```

监视 1

搜索(Ctrl+E)



搜索深度: 3

名称

值

类型



p

0x006ffb30 "hello"



char *

```
a[i] = 0;  
printf("%s \n", a);  
printf("%s \n", p); 已用时间 <= 1ms  
return 0;
```

监视 1

搜索(Ctrl+E)



搜索深度: 3

名称

值

类型



p

0x006ffb30 ""



char *

执行printf("%s\n",a) 后,
p 指向的单元未变
但单元中内容发生变化

Q: 如何实现参数个数不定的函数?

```
printf("....", ..., ...);
```

```
int fsum(int n, ...); 求 n个int类型的数的和
```

```
result= fsum(1, 10);
```

```
result = fsum(2, 10, 20);
```

C程序调用中，传递的入口参数，所占用的存储空间何时释放？

是在子程序中，用 `RET N` 好？
还是回到主程序后，修改 `ESP`，使之指向入口参数之下为好？

Q: 如何实现有默认参数值的函数?

```
void f(int x, int y=10)
{.....}
```

```
f(1,20);
```

```
f(1);
```

在调用函数时，实参个数不足，将默认参数值传入。

Q: 结构类型的参数如何传递？返回一个结构？

```
struct student {  
    int age;  
    int score;  
    int chinese ;  
    int english;  
};  
struct student modify_score(struct student s, int score)  
{  
    s.score = score;  
    return s;  
}  
  
struct student s1;  
struct student s2;  
s2 = modify_score(s1, 100);
```



函数代码的优化

函数编译——代码优化

Debug版本调试中:

在局部变量之上, 留了 40H个字节的空间?

局部变量的初始化值是多少?

保护了未用的一些的寄存器?

Release 版本





函数代码的优化

Release 版本

函数编译——
代码优化

```
:00401010  push 000000C8
:00401015  push 00000064
:00401017  call 00401000
:0040101C  push eax
```

; Possible StringData Ref from Data Obj ->"%d"

```
:0040101D  push 00407030
:00401022  call 00401030
:00401027  add esp, 00000010
:0040102A  xor eax, eax
:0040102C  ret
```

```
:00401000  mov eax, dword ptr [esp+08]
:00401004  mov ecx, dword ptr [esp+04]
:00401008  lea eax, dword ptr [ecx+eax+23]
:0040100C  ret
```

```
sum=fadd(a,b);
printf("%d\n",sum);
```

```
int fadd(int x, int y)
{  int u,v,w;
   u=x+10;
   v=y+25;
   w=u+v;
   return w;
}
```





函数代码的优化

```
:00401000  mov eax, dword ptr [esp+08]
:00401004  mov ecx, dword ptr [esp+04]
:00401008  lea eax, dword ptr [ecx+eax+23]
:0040100C  ret
```

```
int fadd(int x, int y)
{  int u,v,w;
   u=x+10;
   v=y+25;
   w=u+v;
   return w;
}
```

编译优化

未给局部变量分配空间

返回结果在 `eax` 中

进一步优化:

整个函数都可能优化掉



进一步



华中科技大学

- 同一个程序，在不同的开发环境中，在不同的编译开关设置下，编译的结果是不同的！
- 变量和参数的空间分配也是可以变化的！
- 参数的传递方式可以不同（堆栈、寄存器）！
- 参数与寄存器的对应方式可以不同！
- CALL、RET 的执行过程是不变的！





X64 平台下，编译的结果又有什么差别？

```
sum = fadd(a, b);  
00007FF6CABD1908  mov     edx, dword ptr [b]  
00007FF6CABD190B  mov     ecx, dword ptr [a]  
00007FF6CABD190E  call    fadd (07FF6CABD1113h)  
00007FF6CABD1913  mov     dword ptr [sum], eax
```

参数传递的变化：

第一个参数 → ecx

第二个参数 → edx





```
int fadd(int x, int y)
{
```

```
00007FF6CABD1790  mov
00007FF6CABD1794  mov
00007FF6CABD1798  push
00007FF6CABD1799  push
00007FF6CABD179A  sub
00007FF6CABD17A1  lea
00007FF6CABD17A6  lea
00007FF6CABD17AD  call
```

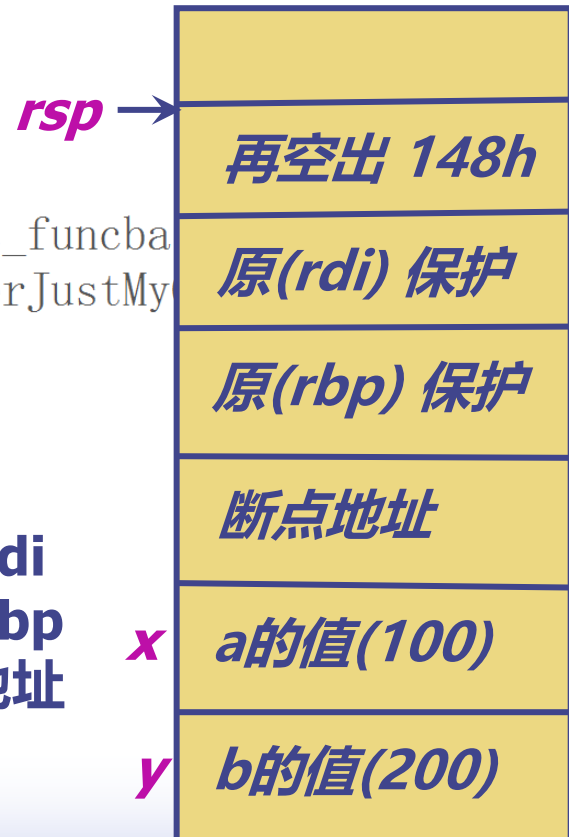
第一个参数: **ecx** 第二个参数: **edx**

```
    dword ptr [rsp+10h], edx
    dword ptr [rsp+8], ecx
    push rbp
    push rdi
    sub rsp, 148h
    lea rbp, [rsp+20h]
    lea rcx, [__8014DB4C_c_funcba
    __CheckForDebuggerJustMy
```



rbp = rsp+20h

rbp+128h -> 保护 rdi
rbp+130h -> 保护 rbp
rbp+138h -> 断点地址
rbp+140h -> a的值
即 x的地址





```
int u, v, w;
```

```
u = x + 10;
```

```
00007FF6CABD17B2  mov
```

```
00007FF6CABD17B8  add
```

```
00007FF6CABD17BB  mov
```

```
eax, dword ptr [rbp+00000000000000140h]
```

```
eax, 0Ah
```

```
dword ptr [rbp+4], eax
```

rsp →

再空出 20h

rbp →

rbp+4 u
rbp+24h v
rbp+44h w

原(rdi) 保护

原(rbp) 保护

断点地址

a的值(100)

b的值(200)

```
v = y + 25;
```

```
mov          eax, dword ptr [rbp+00000000000000148h]
```

```
add          eax, 19h
```

```
mov          dword ptr [rbp+24h], eax
```

```
w = u + v;
```

```
mov          eax, dword ptr [rbp+24h]
```

```
mov          ecx, dword ptr [rbp+4]
```

```
add          ecx, eax
```

```
mov          eax, ecx
```

```
mov          dword ptr [rbp+44h], eax
```





```
int main(int argc, char* argv[])
{
    sub        rsp, 28h
    int  a = 100;    // 0x 64
    int  b = 200;    // 0x C8
    int  sum = 0;
    sum = fadd(a, b);
    printf("%d\n", sum);
    mov        edx, 14Fh
    lea        rcx, [00007FF615D52240h]
    call       00007FF615D51010
    return 0;
    xor        eax, eax
}
add        rsp, 28h
ret
```

X64 平台下,
Release 版
的编译结果

$0x64 + 0xc8 + 0x0A + 0x19$
 $= 0x14F$





```
(gdb) disass /s main
Dump of assembler code for function main:
func_test.c:
14      {
        0x0000000008000676 <+0>:      push    %rbp
        0x0000000008000677 <+1>:      mov     %rsp, %rbp
        0x000000000800067a <+4>:      sub     $0x10, %rsp

15          int a=100;
=> 0x000000000800067e <+8>:      movl    $0x64, -0xc(%rbp)

16          int b=200;
        0x0000000008000685 <+15>:     movl    $0xc8, -0x8(%rbp)

17          int sum=0;
        0x000000000800068c <+22>:     movl    $0x0, -0x4(%rbp)

18          sum=fadd(a, b);
        0x0000000008000693 <+29>:     mov     -0x8(%rbp), %edx
        0x0000000008000696 <+32>:     mov     -0xc(%rbp), %eax
        0x0000000008000699 <+35>:     mov     %edx, %esi
        0x000000000800069b <+37>:     mov     %eax, %edi
        0x000000000800069d <+39>:     callq   0x800064a <fadd>
        0x00000000080006a2 <+44>:     mov     %eax, -0x4(%rbp)
```

Ubuntu 环境中调试 版编译结果

参数在寄存器中
edi a
esi b





华中科技大学

```
(gdb) disass /s fadd
Dump of assembler code for function fadd:
func_test.c:
```

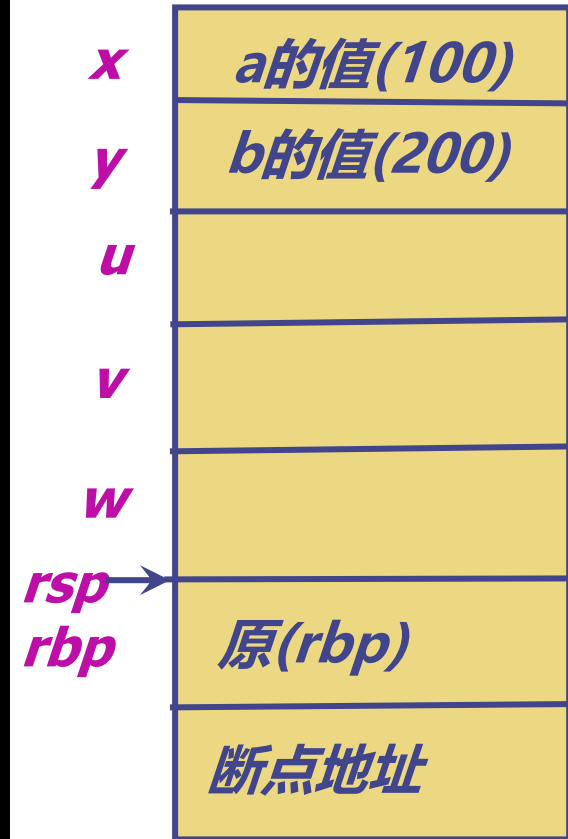
```
3      {
    0x000000000800064a <+0>:      push    %rbp
    0x000000000800064b <+1>:      mov     %rsp,%rbp
    0x000000000800064e <+4>:      mov     %edi,-0x14(%rbp)
    0x0000000008000651 <+7>:      mov     %esi,-0x18(%rbp)

4          int u;
5          int v;
6          int w;
7          u=x+10;
    0x0000000008000654 <+10>:     mov     -0x14(%rbp),%eax
    0x0000000008000657 <+13>:     add     $0xa,%eax
    0x000000000800065a <+16>:     mov     %eax,-0xc(%rbp)

8          v=y+25;
    0x000000000800065d <+19>:     mov     -0x18(%rbp),%eax
    0x0000000008000660 <+22>:     add     $0x19,%eax
    0x0000000008000663 <+25>:     mov     %eax,-0x8(%rbp)

9          w=u+v;
    0x0000000008000666 <+28>:     mov     -0xc(%rbp),%edx
    0x0000000008000669 <+31>:     mov     -0x8(%rbp),%eax
    0x000000000800066c <+34>:     add     %edx,%eax
    0x000000000800066e <+36>:     mov     %eax,-0x4(%rbp)

10         return w;
    0x0000000008000671 <+39>:     mov     -0x4(%rbp),%eax
```



参数

edi a

esi b





X86-64 中, Linux 系统
通过寄存器最多传递 6个整型（即整数 和指针）参数

操作数 大小(位)	参数数量					
	1	2	3	4	5	6
64	%rdi	%rsi	%rdx	%rcx	%r8	%r9
32	%edi	%esi	%edx	%ecx	%r8d	%r9d
16	%di	%si	%dx	%cx	%r8w	%r9w
8	%dil	%sil	%dl	%cl	%r8b	%r9b



小结

- 同一个程序，在不同的开发环境中，在不同的编译开关设置下，编译的结果是不同的！
- 变量和参数的空间分配也是可以变化的！
- 参数的传递方式可以不同（堆栈、寄存器）！
- 参数与寄存器的对应方式可以不同！
- CALL、RET 的执行过程是不变的！





小结

应用程序二进制接口

ABI (Application Binary Interface)

描述了应用程序和操作系统之间、一个应用和它的库之间，或者应用的组成部分之间的接口。

不同的操作系统、32位程序、64位程序，应用程序二进制接口并不相同！





递归函数调用的实现机理

使用递归子程序 求 N!

```
#include <stdio.h>
```

```
int f(int x)
{
    if (x==1) return 1;
    return x*f(x-1);
}
```

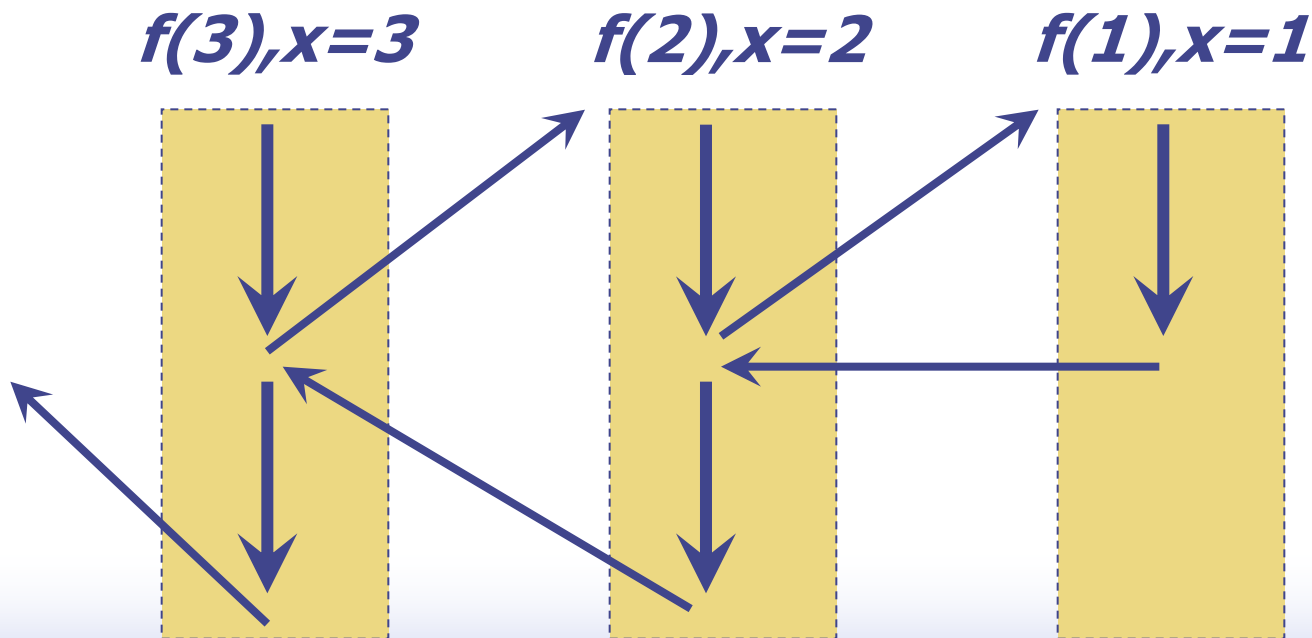
```
void main()
{
    printf("%d\n",f(5));
}
```



递归函数调用的实现机理

递归函数 调用的理解

```
int f(int x)
{
    if (x==1) return 1;
    return x*f(x-1);
}
```

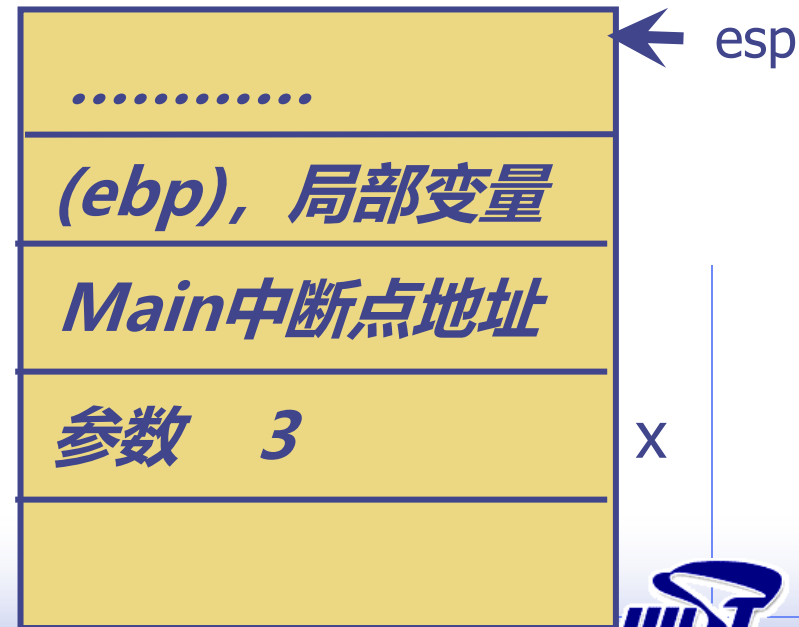
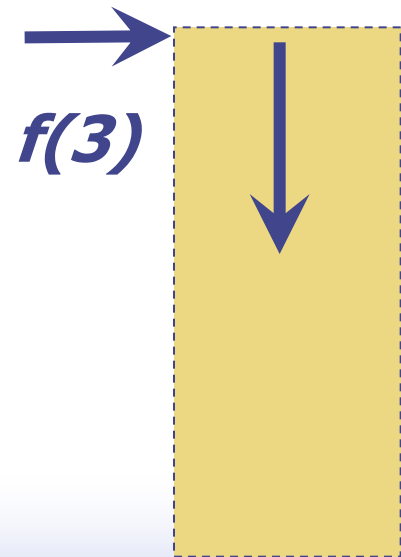




递归函数调用的实现机理

```
int f(int x)
{ if (x==1) return 1;
  return x*f(x-1);
}
```

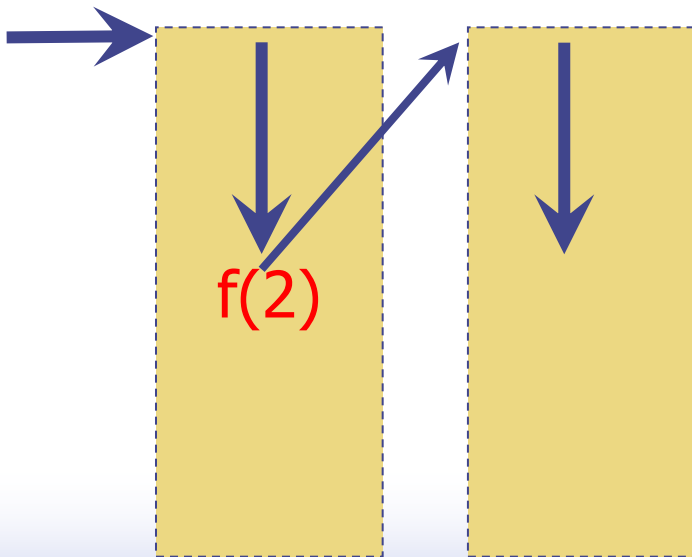
$f(3), x=3$



递归函数调用的实现机理

```
int f(int x)
{
    if (x==1) return 1;
    return x*f(x-1);
}
```

$f(3), x=3$ $f(2), x=2$

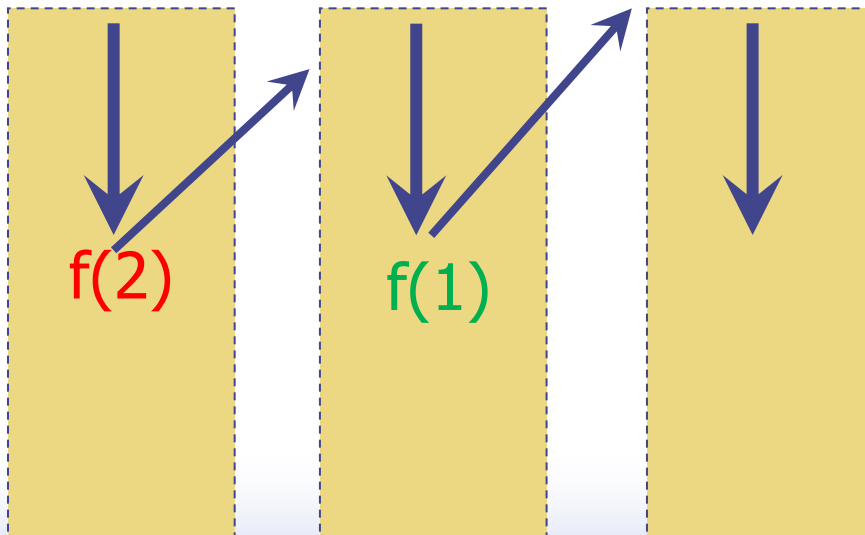




递归函数调用的实现机理

```
int f(int x)
{ if (x==1) return 1;
  return x*f(x-1);
}
```

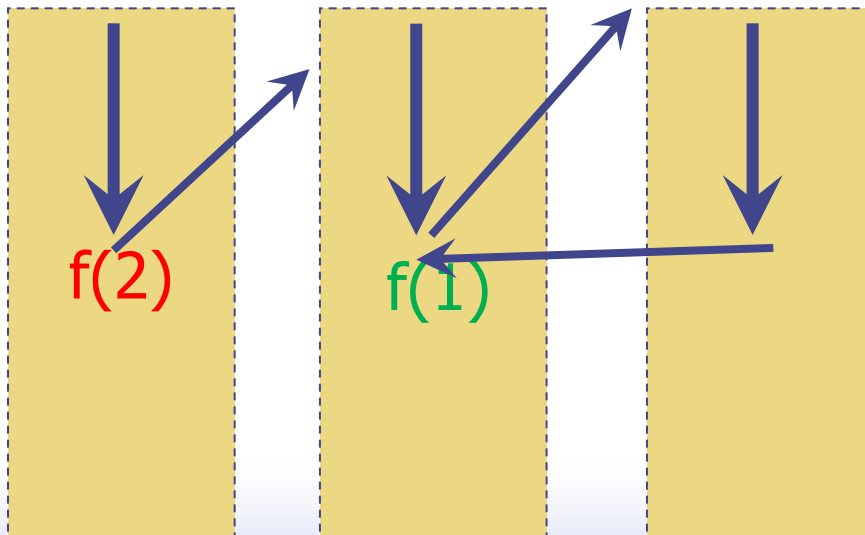
$f(3), x=3$ $f(2), x=2$ $f(1), x=1$



递归函数调用的实现机理

```
int f(int x)
{ if (x==1) return 1;
  return x*f(x-1);
}
```

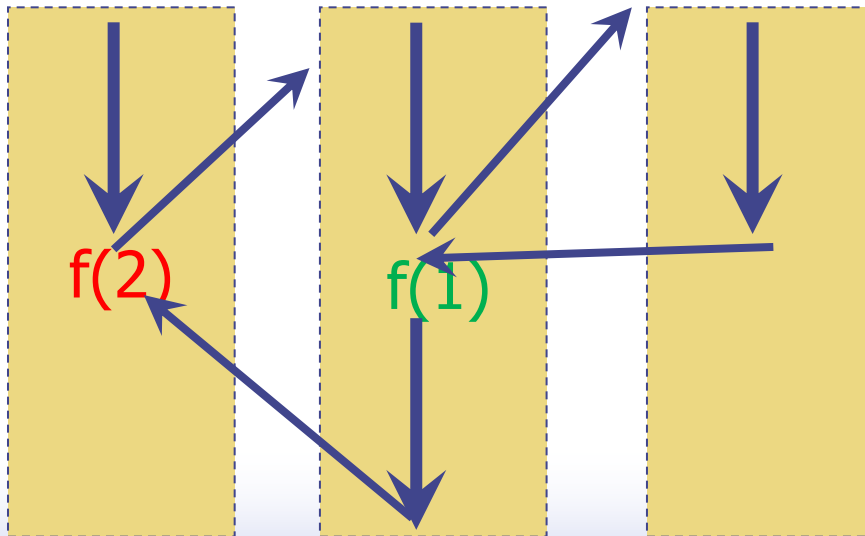
$f(3), x=3$ $f(2), x=2$ $f(1), x=1$



递归函数调用的实现机理

```
int f(int x)
{
    if (x==1) return 1;
    return x*f(x-1);
}
```

$f(3), x=3$ $f(2), x=2$ $f(1), x=1$





递归函数调用的实现机理

递归函数调用

```
:00401000    push esi
:00401001    mov esi, dword ptr [esp+08]
:00401005    cmp esi, 00000001
:00401008    jne 0040100E
:0040100A    mov eax, esi
:0040100C    pop esi
:0040100D    ret

:0040100E    lea eax, dword ptr [esi-01]
:00401011    push eax
:00401012    call 00401000
:00401017    imul eax, esi
:0040101A    add esp, 00000004
:0040101D    pop esi
:0040101E    ret
```

求阶乘的子程序
Release 版本





小结

递归函数的实现机理
递归函数的直观理解
使用递归函数存在的问题



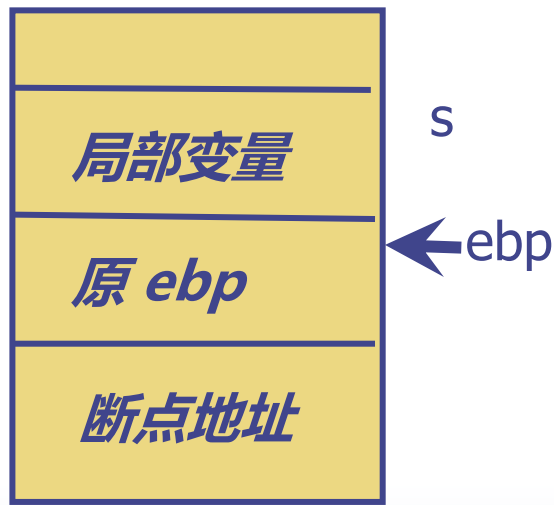


缓冲区溢出

什么是缓冲区溢出？

什么是缓冲区溢出攻击？

如何防范缓冲区溢出攻击？



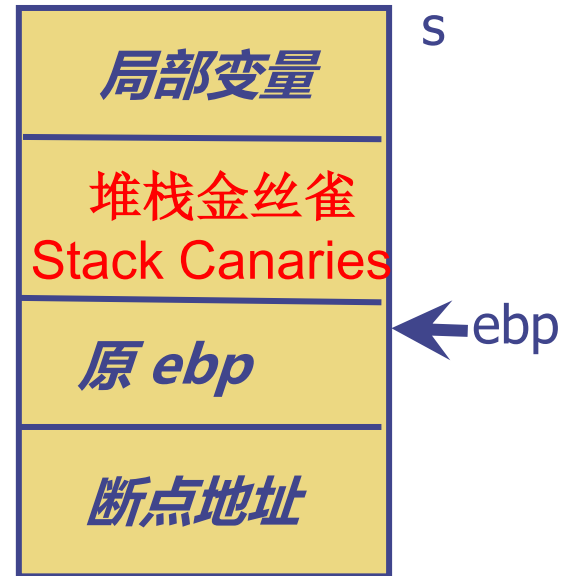
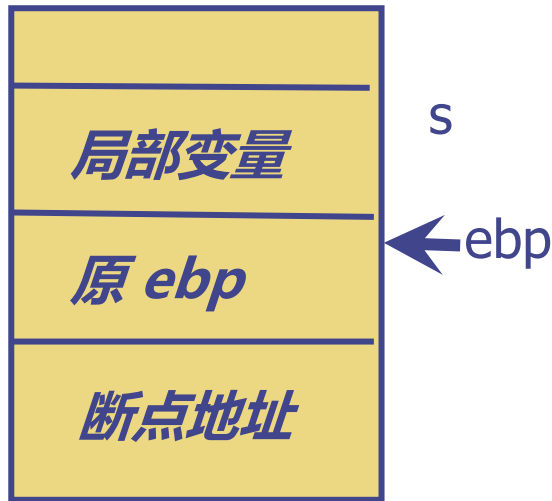
```
char s[10];
```

```
cin >> s;
```

输入串的长度超过 10 个字节，
结果会如何？



缓冲区溢出



增加
哨兵

```
mov    eax,dword ptr [__security_cookie (0730004h)]
xor     eax,ebp          具有随机性的一个数
mov     dword ptr [ebp-4],eax
```

检测
哨兵

```
mov     ecx,dword ptr [ebp-4]
xor     ecx,ebp
call    @__security_check_cookie@4 (0721212h)
```



栈检查

```
int x = 10;
int y = 20;
int* p;
p = &x;
*(p - 1) = 30;
```

已引发异常

Run-Time Check Failure #2 - Stack around the variable 'x' was corrupted.

[复制详细信息](#) | [启动 Live Share 会话...](#)

```
7:      int  x = 10;
00B0256F  mov     dword ptr [ebp-20h], 0Ah
8:      int  y = 20;
00B02576  mov     dword ptr [ebp-2Ch], 14h
```

调试	较小类型检查	否
VC++ 目录	基本运行时检查	两者(/RTC1, 等同于 /RTCsu) (/RTC1)
▲ C/C++	运行库	堆栈帧 (/RTCs)
常规	结构成员对齐	未初始化的变量 (/RTCu)
优化	安全检查	两者(/RTC1, 等同于 /RTCsu) (/RTC1)
预处理器	控制流防护	默认值
代码生成	启用函数级链接	



栈检查

Q: 栈检查的基本原理是什么？



Q: 什么时候进行栈检查？如何进行检查？

函数返回前； 调用一个库函数
call @_RTC_CheckStackVars@8

Q: 栈检查依赖的信息放在什么位置？

在函数最后的ret 之后，开辟了一片数据区，存放要检查的信息。
传递给 RTC_CheckStackVars 有两个参数，包括栈的指针，以及数据区的指针。





栈检查

```
func proc
    push    ebp
    mov     ebp, esp
    sub     esp, 24h          ; 分配栈空间（含变量和填充）
    ; ... 函数逻辑 ...
```

```
    lea     eax, [RTC_VarInfo] ; 加载元数据地址
    push    eax                ; 参数1: 元数据
    lea     ecx, [ebp-24h]      ; 参数2: 栈帧基地址
    call    _RTC_CheckStackVars@8 ; 检查栈变量
    mov     esp, ebp
    pop     ebp
    ret
```

```
RTC_VarInfo DD .....
```

```
func endp
```





未初始化的变量使用检测

```
int x = 10;
int z;
int y;
cout<< y; //
if (x > 10)
    z = 30;
cout << "z=" << z << endl;
```

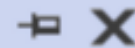
Q: 程序有什么问题?

编译报错:

使用了未初始化的局部变量 `y`;
但对于 `z` 没有报错.

Q: 去掉 `cout<<y`; 程序运行有什么问题?

已引发异常



Run-Time Check Failure #3 - The variable 'z' is being used without being initialized.

Q: 如何检测出这一错误的?





总结

CALL、RET

参数传递的方法

参数、局部变量的空间分配与回收

参数、局部变量的地址表达形式

寄存器的保护和恢复

栈帧结构

递归调用程序的理解

缓冲区溢出、攻击及防范

栈检查实现的机理

